Project Proposal: DevExplorer

Overview

This project presents the design and implementation of a Minimal Developer-Oriented File Explorer for Windows, built upon a custom hybrid Abstract Data Type (ADT). The system combines traditional file navigation with advanced search and developer-focused utilities, addressing limitations present in the native Windows File Explorer.

Unlike conventional GUI-driven projects, this system prioritizes data-structure-driven design, where all core functionality is encapsulated within a hybrid ADT composed of trees, tries, hash maps, inverted indexes, and priority queues. A lightweight graphical interface is layered on top of this ADT to demonstrate usability without obscuring the underlying logic.

1) Problem Statement

Windows File Explorer is optimized for general users but lacks features, critical for developer workflows, such as fast filename searching, content-based search across codebases, extension-based filtering, and context-aware command execution. Developers often rely on multiple external tools to compensate for these shortcomings, leading to fragmented workflows and inefficiency.

Linux environments natively provide such capabilities through built-in commands and open-source utilities. There is a clear need for a unified, lightweight, and developer-focused file explorer on Windows that delivers these features while remaining simple, fast, and explainable.

2) Hybrid ADT Design

The system is implemented as a hybrid ADT, integrating multiple data structures to achieve optimal performance and modularity. A tree structure models the directory hierarchy, enabling intuitive navigation. Filenames are indexed using a Trie to support instant prefix-based searching. File contents are indexed using an inverted index implemented via hash maps and posting lists, enabling efficient full-text search.

File metadata is stored in hash maps for constant-time access, while search results are ranked using a priority queue based on term frequency. All indexing and filesystem monitoring operations are handled by a single background worker thread, while user interaction and search queries are handled by the UI thread, enforcing a strict two-thread execution model.

3) Hybrid ADT Operations

Navigation Operations

These operations define how the ADT models and navigates the filesystem hierarchy.

- openDirectory(path)
- navigateToParent()
- navigateToPath(path)
- listCurrentDirectory()
- getCurrentPath()
- refreshCurrentDirectory()

Search Operations

These operations treat search as a **navigation primitive**, not a separate tool.

- searchByFilenamePrefix(prefix)
- searchByFileContent(query)
- searchInCurrentDirectory(query)
- filterByExtension(extension)
- rankSearchResults(results)
- clearSearch()

Indexing Operations

These operations define the **lifecycle of the hybrid index**, which is central to the ADT.

- buildInitialIndex(rootPath)
- updateIndexOnFileCreate(filePath)
- updateIndexOnFileModify(filePath)
- updateIndexOnFileDelete(filePath)
- reindexDirectory(path)
- loadIndexFromDisk()
- persistIndexToDisk()

Developer Utility Operations

These operations provide **Linux-like developer conveniences** missing in Windows Explorer.

- openTerminalHere()
- copyAbsolutePath(file)
- copyRelativePath(file)
- toggleCodeMode(enable)
- applyIgnoreRules(patterns)

System & Concurrency Control Operations

These operations formalize the **two-thread execution model**.

- startBackgroundIndexer()
- stopBackgroundIndexer()
- handleFileSystemEvent(event)
- shutdown()